Практичне заняття №5

Тема. **Обробка виняткових ситуацій. Застосування блоків try і catch**
Мета і завдання практичного заняття: розгляд виняткових ситуацій та механізму їх обробки; набуття практичних навичок розробки програм мовою Java у середовищі IntelliJ IDEA, в яких передбачено обробку виключних ситуацій; залучити до самостійної діяльності та прийняття рішень при формуванні програмного коду.

Питання для перевірки базових знань за темою практичного заняття:

1. Що таке парадигма "виключення"?
2. Яка ієрархія виключень?
3. Можна / чи потрібно обробляти помилки jvm?
4. Які існують способи обробки виключень?
5. Про що говорить ключове слово throws?
6. У чому особливість блоку finally? Чи завжди він виконується?
7. Чи може не бути жодного блоку catch при відлові виключень?
8. При якій ситуації блок finally не буде виконано?
9. Чи може один блок catch відловлювати кілька виключень (з одного і різних гілок наслідувань)?
10. Що таке оброблювані і необроблювані (checked / unchecked) виключення?
11. У чому особливість RuntimeException?
12. Як написати власне ("призначене для користувача") виключення?
13. Який оператор дозволяє примусово викинути виключення?
14. Чи є додаткові умови до методу, який потенційно може викинути виключення?
15. Якщо оператор return міститься і в блоці catch, і в finally, який з них "головніший"?
16. Яке призначення OutOfMemoryError?
17. Яке призначення SQLException? До якого типу checked або unchecked воно відноситься, чому?
18. Що таке Error? В якому випадку використовується Error?
19. Яка конструкція використовується в Java для обробки виключень?
20. Припустимо, є блок try-finally. У блоці try виникло виключення і виконання перемістилося в блок finally. У блоці finally теж виникло виключення. Яке з двох виключень "випаде" з блоку try-finally? Що трапиться з другим виключенням?

21. Припустимо, є метод, який може викинути IOException і FileNotFoundException в якій послідовності повинні йти блоки catch? Скільки блоків catch буде виконано?
22. Якими способами можуть генеруватись виключення?
23. В яких випадках викликається стандартний обробник виключень Java?
24. Які ключові слова використовуються для обробки виключень в Java?
25. Яке призначення конструкції try… catch…finally?
26. Яке призначення ключового слова throw?
27. Які типи заборонено використовувати для генерування виключень оператором throw?
28. Яким чином створити власний клас виключення?

Завдання:

Завдання 1.

1. Написати програму мовою Java, в якій відповідно до заданого варіанту:

- визначити експериментально, помилки яких класів будуть згенеровані;
- створити обробники виняткових ситуацій з використанням виявлених класів і всіх секцій конструкції обробника з відповідними повідомленнями, що дозволяють коректно виконати програму.

Завдання виконати у вигляді двох проектів: без використання власних методів і з використанням методів для кожної підзадачі, які можуть генерувати виключну ситуацію.

Варіант 1

У програмі, що обчислює середнє значення серед додатних елементів одновимірного масиву (тип елементів int), який вводиться з клавіатури, можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- невідповідність числового типу даних;
- додатні елементи відсутні;
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 2

У програмі, де потрібно з матриці вивести стовпець з номером, заданим з клавіатури, можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- немає стовпчика з таким номером.
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 3

У програмі, що обчислює суму елементів типу byte одновимірного масиву, який вводиться з клавіатури, можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- введення або обчислення значення за межами діапазону типу.
- - передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 4

У програмі, де потрібно для квадратної матриці з елементами, введеними з клавіатури, вивести максимальний парний елемент на бічній діагоналі, можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- немає парних чисел.
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 5

У програмі, що обчислює мінімум серед від'ємних елементів одновимірного масиву (тип елементів float) і номер цього мінімуму, можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- від'ємні елементи відсутні.
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 6

У програмі, де потрібно з матриці вивести рядок з номером, заданим з клавіатури, можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- немає такого номера рядка.
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 7

У програмі пошуку максимального значення серед додатніх елементів в одновимірному масиві (тип елементів byte) і номери цього максимуму можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- введення або обчислення значення за межами діапазону типу;
- відсутні додатні елементи;
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 8

У програмі, що обчислює середнє значення серед від'ємних елементів одновимірного масиву, який вводиться з клавіатури (тип елементів int), можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- невідповідність числового типу даних;
- від'ємні елементи відсутні.
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 9

У програмі, де потрібно з матриці з елементами, введеними з клавіатури, вивести рядок з мінімальним від'ємним числом, можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- немає від'ємних чисел.
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 10

У програмі, де в одновимірному масиві, який вводиться з клавіатури, обчислюється максимум серед чисел, що діляться на 3, і номер максимуму, можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- відсутні елементи, які діляться на 3.
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 11

У програмі, де для квадратної матриці з елементами, введеними з клавіатури, потрібно вивести максимум серед парних чисел на головній діагоналі, можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- немає парних чисел.
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 12

У програмі, де потрібно з матриці з елементами, введеними з клавіатури, вивести стовпець з максимальним парним числом, можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- невідповідність числового типу даних;
- немає парних чисел.
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 13

У програмі, де потрібно перемножити дві довільні матриці, можуть виникати помилки в наступних випадках:

- невідповідність розмірності матриць;
- невідповідність числового типу даних;
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 14

У програмі, що обчислює середнє значення серед додатних елементів одновимірного масиву (тип елементів int), який вводиться з клавіатури, можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- невідповідність числового типу даних;
- додатні елементи відсутні;
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 15

У програмі, де потрібно з матриці вивести стовпець з номером, заданим з клавіатури, можуть виникати помилки в наступних випадках: - введення рядка замість числа;

- немає стовпчика з таким номером.
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 16

У програмі, що обчислює суму елементів типу byte одновимірного масиву, який вводиться з клавіатури, можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- введення або обчислення значення за межами діапазону типу. - передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 17

У програмі, де потрібно для квадратної матриці з елементами, введеними з клавіатури, вивести максимальний парний елемент на бічній діагоналі, можуть виникати помилки в наступних випадках: - введення рядка замість числа; - немає парних чисел.

- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 18

У програмі, що обчислює мінімум серед від'ємних елементів одновимірного масиву (тип елементів float) і номер цього мінімуму, можуть виникати помилки в наступних випадках: - введення рядка замість числа; - від'ємні елементи відсутні.

- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 19

У програмі, де потрібно з матриці вивести рядок з номером, заданим з клавіатури, можуть виникати помилки в наступних випадках: - введення рядка замість числа; - немає такого номера рядка.

- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 20

У програмі пошуку максимального значення серед додатніх елементів в одновимірному масиві (тип елементів byte) і номери цього максимуму можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- введення або обчислення значення за межами діапазону типу; - відсутні додатні елементи.
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 21

У програмі, що обчислює середнє значення серед від'ємних елементів одновимірного масиву, який вводиться з клавіатури (тип елементів int), можуть виникати помилки в наступних випадках:

- введення рядка замість числа; - невідповідність числового типу даних; - від'ємні елементи відсутні.
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 22

У програмі, де потрібно з матриці з елементами, введеними з клавіатури, вивести рядок з мінімальним від'ємним числом, можуть виникати помилки в наступних випадках:

- введення рядка замість числа; - немає від'ємних чисел.
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 23

У програмі, де в одновимірному масиві, який вводиться з клавіатури, обчислюється максимум серед чисел, що діляться на 3, і номер максимуму, можуть виникати помилки в наступних випадках: - введення рядка замість числа;

- відсутні елементи, які діляться на 3.

- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 24

У програмі, де для квадратної матриці з елементами, введеними з клавіатури, потрібно вивести максимум серед парних чисел на головній діагоналі, можуть виникати помилки в наступних випадках: - введення рядка замість числа; - немає парних чисел.

- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 25

У програмі, де потрібно з матриці з елементами, введеними з клавіатури, вивести стовпець з максимальним парним числом, можуть виникати помилки в наступних випадках:

- введення рядка замість числа;
- невідповідність числового типу даних;
- немає парних чисел.
- передбачити інші виняткові ситуації, які

генеруються програмою.

Варіант 26

У програмі, де потрібно перемножити дві довільні матриці, можуть виникати помилки в наступних випадках:

- невідповідність розмірності матриць;
- невідповідність числового типу даних;
- передбачити інші виняткові ситуації, які генеруються програмою.

Варіант 27

У програмі, де потрібно перемножити два вектори між собою, можуть виникати помилки в наступних випадках:

- невідповідність розмірності векторів;
- невідповідність числового типу даних;
- передбачити інші виняткові ситуації, які генеруються програмою.

Теоретичні відомості та методичні вказівки для виконання

1. Виключення (exceptions). Виключна ситуація.

Виключна ситуація – це програмна помилка, яка виникає під час виконання послідовності програмного коду. Програмна помилка може бути логічною помилкою програміста під час розробки програми.

Наприклад, виключна ситуація може виникати у випадку:

- спроби ділення на нуль
- спроби звернення до елементу масиву, номер індексу якого виходить за межі оголошеного;
- спроби взяти корінь з від'ємного числа;
- спроби відкрити файл за іменем, якого немає на диску;
- інші випадки.
- Виділяють три типи виключень.

Контрольовані виключення - представляють собою вид виключення, яке відбувається на стадії компіляції, їх також називають виключеннями періоду компіляції. Окреслені виключення не слід ігнорувати в ході компіляції, вони вимагають належного управління (дозволу) з боку програміста.

Неконтрольовані виключення - є виключеннями, які відбуваються під час виконання. Вони також носять назву виключення на етапі виконання. Дана категорія може включати помилки програмування, такі як логічні помилки або невірний спосіб використання API. Виключення на етапі виконання ігноруються в ході компіляції.

Помилки - не є виключеннями, проте представляють проблеми, які виникають незалежно від користувача або програміста. Помилки в коді зазвичай ігноруються тому, що в рідкісних випадках їх обробка виявиться результативною. Наприклад, помилка виникне при переповненні стеку. На етапі компіляції вони також ігноруються.

Правильна обробка виключень є важливим елементом написання професійних програм на Java.

Як правило, ***кожна виключна ситуація має мати свій код помилки*** та обробку цього коду (виведення відповідних повідомлень, тощо). У мові програмування Java розроблено механізм обробки виключних ситуацій.

У мові програмування Java, виключення – це спеціальний об'єкт, який описує виключну ситуацію, що виникла в деякій частині програмного коду. Об'єкт, що представляє виключення, генерується в момент виникнення виключної ситуації. Після виникнення критичної (виключної) ситуації виключення перехоплюється та обробляється. Таким чином, виникає поняття

генерування виключення. Генерування виключення – процес створення об’єкту, що описує дане виключення.

У мові Java виключення можуть генеруватись одним з двох способів:

- ***автоматично***. У цьому випадку виключення генеруються виконавчою системою Java. До таких виключень входять помилки, що порушують правила мови Java чи обмеження, що накладаються системою;
- ***вручну***. У цьому випадку виключення генеруються в програмному коді, який розробляється. Ручні виключення програмуються для повідомлення викликаючому коду про можливі помилки в методах розроблюваної програми.

Стандартний обробник виключень Java викликається, коли програма:

- не використовує блок try…catch для обробки та перехоплення виключної ситуації взагалі;
- містить блок try…catch, однак у цьому блоці даний тип виключення не перехоплюється.

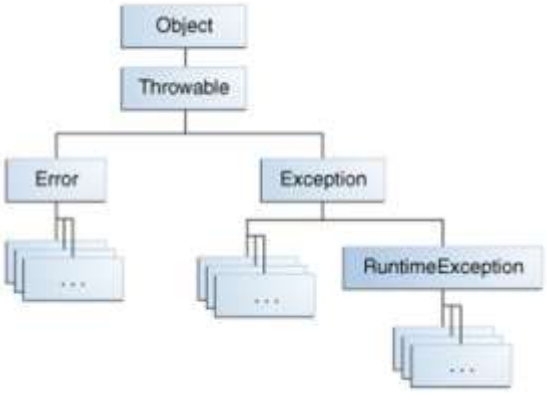
Якщо програма містить власний код try…catch для обробки даної виключної ситуації, тоді стандартний обробник виключень не викликається.

Для обробки виключень в Java використовуються наступні ключові слова:

- try;
- catch;
- throw;
- throws;
- finally.

2. Ієрархія виключень

Всі класи виключень в Java представляють підтипи класу java.lang.Exception. Клас виключень є підкласом класу Throwable. Крім класу виключень існує також підклас помилок, що утворилися з класу Throwable.





Помилки представляють аварійний стан внаслідок значних збоїв, які не обробляються програмами Java. Генерування помилок призначене для відображення помилок, виявлених середовищем виконання. Приклади: JVM вичерпав наявний обсяг пам'яті. Зазвичай, програми не можуть відновити неполадки, викликані помилками.

Клас виключень ділиться на два основних підкласи: клас IOException і клас RuntimeException. Всі виключення діляться на три групи:

Виключення (Exceptions) є результатом проблем в програмі, які в принципі можна вирішити і передбачити. Наприклад, відбулося ділення на нуль в цілих числах.

Помилки (Errors) представляють собою більш серйозні проблеми, які, відповідно до специфікації Java, не слід намагатися обробляти у власній програмі, оскільки вони пов'язані з проблемами рівня JVM. Наприклад, виключення такого роду виникають, якщо закінчилася пам'ять, доступна віртуальній машині. Програма додаткову пам'ять все одно не зможе забезпечити для JVM.

У класу Exception є важливий підклас - RuntimeException. Виключення цього типу автоматично визначаються для створюваних прикладних програм і охоплюють такі помилки, як ділення нуль і помилкова індексація масивів.

Також виключення ***діляться за типом*** на неконтрольовані і контрольовані виключення. ***Неконтрольовані (unchecked)*** - RuntimeException,
Error і їх спадкоємці.

Контрольовані виключення (checked) - всі інші спадкоємці класу Exception.

3. Ключові слова try, catch, finally

У мові Java можна вручну перехоплювати виключення (виключні ситуації) та відповідним чином обробляти їх. Важливо, щоб при виникненні виключної ситуації програма не припинила своє виконання.

Для відслідковування, генерування та обробки виключень у мові програмування Java використовується конструкція ***try…catch…finally***, яка має таку загальну форму

```
try {
    // блок коду, в якому відслідковуються та генеруються виключення
    // ...
}
catch (type_exception_1 objEx1) {
    // обробник виключення типу type_exception_1
    // ...
}
catch (type_exception_2 objEx2) {
    // обробник виключення типу type_exception_2
    // ...
}
...
catch (type_exception_N objExN) {
    // обробник виключення типу type_exception_N
    // ...
}
finally {
    // блок коду, що має бути обов'язково виконаний
    // після завершення блоку try
    // ...
}
```

У ***блоці try*** розміщуються оператори програми, які потрібно проконтролювати та у випадку виникнення виключної ситуації згенерувати виключення. Всередині блоку try можуть викликатись різні методи, здатні згенерувати те чи інше виключення. Однак, обробник виключення буде тільки один.

У ***блоці catch*** розміщується програмний код, який обробляє перехоплене виключення (обробник виключення). Код у блоці catch реалізує виконання відповідних дій, якщо відбулась виключна ситуація у блоці try. Блоків catch може бути декілька. Якщо генерується виключення, механізм обробки виключень шукає перший з обробників виключень, аргумент якого відповідає поточному типу виключення. Після цього він входить у блок catch, і, в результаті виключення вважається обробленим. Тільки один відповідний блок catch обробляється.

У ***блоці finally*** вказується код, який повинен бути обов'язково виконаний після завершення блоку try. Блок операторів finally виконується незалежно від того, чи буде згенероване виключення чи ні. Якщо виключення згенероване, блок операторів finally виконується навіть при умові, що жоден з операторів catch не співпадає з цим виключенням.

Оператори try і catch складають єдине ціле. Оператор finally може бути відсутній.



Приклад генерування виключення та його перехоплення стандартним обробником виключень Java

Якщо у програмі відсутній власний блок try…catch перехоплення виключення, викликається стандартний обробник виключення. У прикладі демонструється виклик стандартного обробника.

Оголошується клас DemoExceptions. З метою демонстрації у класі оголошується метод DivNumbers(). Метод повертає число типу double, що є результатом ділення двох цілих значень змінних a та b. Ці змінні є вхідними параметрами методу.

Якщо b = 0 то при діленні a на b може виникнути виключна ситуація “ділення на нуль” типу ArithmeticException. У програмі немає коду, який явно перехоплює та обробляє цю ситуацію. Тому, викликається стандартний обробник Java.

Текст класу DemoExceptions наступний:

```
// клас, що реалізує метод,
// в якому може виникнути виключна ситуація
class DemoExceptions {
    // метод, в якому може виникнути ділення на 0
    double DivNumbers(int a, int b) {
        // при діленні на 0 викликається стандартний обробник Java
        double res;
        res = a/b;
        return res;
    }
}
```

Якщо в іншому класі використати метод DivNumbers() наступним чином

```java
public class Train01 {

    public static void main(String[] args) {
        DemoExceptions dE = new DemoExceptions(); // створити екземпляр класу
        double res;

        // викликати метод DivNumbers()
        res = dE.DivNumbers(2, 0); // 2/0 => ділення на нуль
        System.out.println("res = " + res);
    }
}
```